



KOOR

The Somali Programming Language

"The Complete Beginner's Guide: Zero-Coding Foundations, Download & Setup, Terminal Reference, the 6 Core CS Pillars, and 4 Hands-On Real-World Projects."

• Zero-Coding Foundations

• Download & Setup Guide

• Terminal CLI Commands

• The 6 Core CS Pillars

• 4 Real-World Projects

FOUNDATIONS OF PROGRAMMING IN KOOR

"This book is dedicated to aspiring developers, students, and educators. By grounding foundational computer science concepts in authentic, natural Somali grammar, Koor empowers anyone to master computational thinking without linguistic barriers."

— Koor Core Documentation Team

1. WELCOME TO CODING Zero-Coding Foundations, Algorithms, & The Koor Interpreter	2. DOWNLOAD & SETUP Installation Guide, PATH Configuration, & VS Code Setup
3. TERMINAL COMMANDS Complete CLI Reference (koor run, repl, check)	4. VARIABLES & TYPES Memory Containers & Primitive Data Types (tiro, qoraal)
5. OPERATORS & LOGIC Arithmetic & Natural Comparison Expressions (sidoo kale)	6. INPUT & OUTPUT Screen Output (waxaad soo saartaa) & Keyboard Input (waydiin)
7. CONTROL FLOW Decisions (haddii, kale) & Automated Loops (ku celi, inta)	8. FUNCTIONS Reusable Modules (hawl) & Return Values (waxaad soo celisaa)
9. DATA STRUCTURES Lists (liis) & Key-Value Dictionaries (qaamuus)	10. REAL-WORLD PROJECTS 4 Built Projects (Retail POS, Student Grading, EVC+, Guessing Game)

Mastering Computer Science Through Natural Somali

This handbook is crafted specifically for absolute beginners who have never programmed before. By working through clear analogies, step-by-step setup tutorials, interactive exercises, and real-life projects, you will develop genuine problem-solving skills that apply to any modern programming language.

The Comprehensive Guide to Natural Somali Programming, Setup, CLI, the Six Core Pillars & Real-World Projects

TABLE OF CONTENTS

Table of Contents

01Chapter 1: Welcome to Programming & Koor

- 1.1 What is Programming? (Zero-Coding Foundations)
- 1.2 How Computers Understand Code (The Execution Pipeline)
- 1.3 Why Koor? The Power of Natural Somali Programming
- 1.4 How Statements Work & Why Memory Differs from Screen Output

02Chapter 2: Download & Setup Guide

- 2.1 System Prerequisites (Windows, macOS, Linux)
- 2.2 Downloading Koor from Source
- 2.3 One-Click Windows Installer (`KoorSetup.exe`)
- 2.4 Portable Executable & Built-In `koor install`
- 2.5 Setting Up Visual Studio Code & Your First File

03Chapter 3: Complete Terminal Commands

- 3.1 Demystifying the Command Line for Total Beginners
- 3.2 Running Koor Files: `koor run`
- 3.3 The Interactive Playground: `koor repl`
- 3.4 Validating Syntax: `koor check`
- 3.5 Built-In Help Reference: `koor --help`
- 3.6 Essential Terminal Shortcuts & Tips

04Chapter 4: Pillar 1 — Variables & Data Types

- 4.1 What is a Variable? The Memory Container Concept
- 4.2 Three Ways to Declare Variables (`waa` , `qeex ... inuu` , `=`)
- 4.3 Variable Identifiers & The Somali Glottal Apostrophe (`'`)
- 4.4 Variables vs. String Literals (`x` vs. `"x"`)
- 4.5 Natural Verbal In-Place Modifications (`da' waxaad ku dartaa 5` , `waxaad 1 ku dartaa tirsade`)
- 4.6 The 4 Primitive Data Types (`tiro` , `qoraal` , `run-been` , `waxba`)
- 4.7 Type Inspection & Explicit Type Casting (`nooc` , `tiro()` , `qoraal()`)

05Chapter 5: Pillar 2 — Operators & Expressions

- 5.1 Arithmetic Operators & Natural Zero-Signs Math (`ku dar` , `ka jar` , `ku dhufo` , `u qaybi` , `haraaga`)
- 5.2 Natural Somali Comparison Phrases vs. Symbolic Operators
- 5.3 Gender Agreement in Comparisons (`uu` vs. `ay`)
- 5.4 Logical Operators (`sidoo kale` , `iyo` , `ama` , `ma` , `ma aha`)

06Chapter 6: Pillar 3 — Input and Output (I/O)

- 6.1 Output Commands: `waxaad soo saartaa` , `daabac` , `qor` , `soo saar`
- 6.2 Printing Variables, Literals, and Comma-Separated Values
- 6.3 Dynamic String Interpolation (`{variable}` and `{object.field}`)
- 6.4 Interactive Keyboard Input with `waydiin()`
- 6.5 Reading Numbers: `tiro(waydiin( ... ))` is Mandatory Why

07Chapter 7: Pillar 4 — Control Flow (Decisions & Loops)

- 7.1 Indentation Rules, Colons (`:`), and Code Blocks
- 7.2 Single-Line Spoken Conditionals
- 7.3 Multi-Branch Conditionals (`haddii` , `hadii kale oo ay / uu` , `kale`)
- 7.4 Truthiness Rules in Koor (Truthy vs. Falsy Values)
- 7.5 Fixed-Count Loops (`ku celi ... jeer`)
- 7.6 Conditional While Loops (`inta`)
- 7.7 Collection Iteration (`mid kastoo ku jira (qof)`)
- 7.8 Loop Jump Controls: Break (`ka bax`) and Continue (`ka bood`)

08Chapter 8: Pillar 5 — Functions (Hawlaha)

- 8.1 Defining Block Functions (`hawl`)
- 8.2 Returning Values (`waxaad soo celisaa`) vs. Printing (`daabac`)
- 8.3 Structured Declarative Functions (`magaceed waa` , `tibxuhu waa` , `kaydi natiijada`)
- 8.4 Function Invocation with `qabo hawshan` (`qabo hawshan (hawl) (doodo)`)
- 8.5 Parameters, Arguments, and Local Scope
- 8.6 Recursive Functions (Functions Calling Themselves)

09Chapter 9: Pillar 6 — Data Structures

- 9.1 Ordered Lists (`liis`): Indexing, Zero-Based Access, & Mutation
- 9.2 Built-In List Methods (`.ku_dar()` , `.ka_saar()` , `.kala_sooc()` , `.rog()` , `dherer()`)
- 9.3 Key-Value Dictionaries (`qaamuus`): Bracket & Dot Notations
- 9.4 Text Sequences & String Methods (`.jar()` , `.weyneey()` , `.yaree()` , `.beddel()` , `.kala_bax()`)
- 9.5 Built-In Mathematical & Utility Functions (`nasiib` , `xidid` , `qaanso`)

10Chapter 10: Real-World Projects (Mashaariic Dhab Ah)

- 10.1 Project 1: Retail Store POS & Invoice Generator
- 10.2 Project 2: Student Grading & Class Analytics System
- 10.3 Project 3: Mobile Money & Digital Banking Wallet (EVC Plus / Zaad)
- 10.4 Project 4: Interactive Number Guessing Game (With Hints & Attempts)

Welcome to Programming & Koor

If you have never written a single line of code in your life, or feel intimidated by computer science jargon, **welcome! You are in the right place.** This chapter builds your foundation from absolute ground zero, assuming no prior technical background whatsoever.

1.1 What is Programming? (Zero-Coding Foundations)

Computers are remarkably fast and capable machines, but **they cannot think on their own.** A computer has no independent consciousness, intuition, or common sense. Left alone, it cannot perform any task without an unambiguous, step-by-step list of instructions written by a human.

The Recipe Analogy (How Code Works):

Imagine giving instructions to someone who has never cooked before on how to prepare traditional Somali flatbread (Canjeero or Sabaayad):

1. Take a large mixing bowl.
2. Measure 2 cups of flour and 1 cup of lukewarm water.
3. Stir continuously for 5 minutes until smooth.
4. Pour one ladle of batter onto the hot pan.

That ordered sequence of steps is called an **Algorithm**. When you express an algorithm in a precise formal syntax that a computer can read and execute, your instructions are called **Code**, and the craft of writing them is called **Programming**.

1.2 How Computers Understand Code (The Execution Pipeline)

When you write a program in Koor, you simply create a standard text file whose name ends with the `.koor` extension (such as `hello.koor`).

How does your text turn into actions on the computer screen?

1. **The Source File (`.koor`):** A plain text file containing your human-readable instructions written in authentic Somali syntax.
2. **The Koor Interpreter:** An engine on your computer that reads your instructions line by line, checks them for grammar rules, translates them into low-level machine instructions, and executes them.
3. **The Standard Output:** The terminal screen where results, messages, calculations, and answers are displayed back to you.

1.3 Why Koor? The Power of Natural Somali Programming

Almost every widely used programming language in the world—such as Python, Java, or C++—is designed around English vocabulary (`if`, `while`, `function`, `print`). For native Somali speakers, this imposes a double burden:

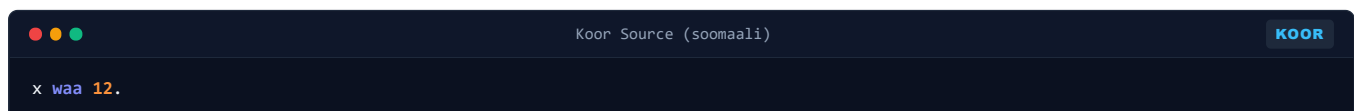
- You have to learn challenging computer science and mathematical concepts.
- You simultaneously have to navigate foreign English idioms and terminology.

Koor eliminates this barrier. It is engineered from the ground up around **authentic natural Somali grammar**. Instead of cryptic foreign phrases, you state facts equatively using `waa` (*is*), output information using `waxaad soo saartaa` (*bring forth / print*), make decisions using `haddii` (*if*), and package reusable actions into `hawl` (*function*).

1.4 How Statements Work & Why Memory Differs from Screen Output

One of the most common surprises for beginners is running a program and seeing **nothing on the screen**.

Consider this code:



```
Koor Source (soomaali) KOOR
x waa 12.
```

If you run this file with `koor run test.koor`, the terminal will display **no output at all!** Why?

- Because storing a value in a variable (`x waa 12.`) is an **internal memory operation**. You instructed the computer to put the number 12 into a memory slot named `x`.
- Storing something in memory does **not** print it to the screen. The computer quietly completes the task and exits.
- If you want to see something on the screen, you must give an explicit **output instruction**, such as `daabac x` or `waxaad soo saartaa x`.

Download & Setup Guide

Before you start writing code, let's get Koor installed and configured on your machine. Follow this straightforward step-by-step guide.

2.1 System Prerequisites (Windows, macOS, Linux)

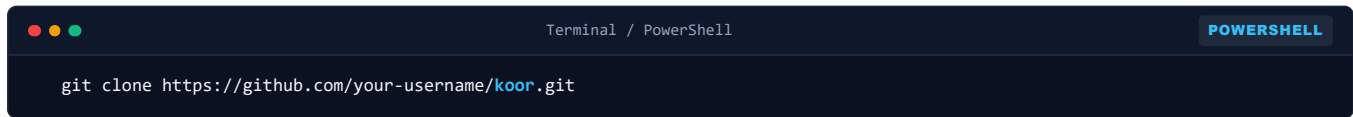
Koor is lightweight and cross-platform:

- **Operating System:** Windows 10/11, macOS, or any Linux distribution.
- **Python Runtime:** Python 3.10 or newer (downloadable free of charge from python.org).
- *Windows Standalone Note:* Windows users can also run the precompiled `koor.exe` executable, which runs standalone without requiring Python.

2.2 Downloading Koor from Source

You can obtain Koor directly from its repository:

1. Download the project repository as a ZIP archive or clone it using Git:



```
git clone https://github.com/your-username/koor.git
```

2. Extract or move the folder to a convenient location on your drive, such as `C:\Koor` or `D:\zdiiv\new coding language`.

2.3 One-Click Windows Installer (`KoorSetup.exe`)

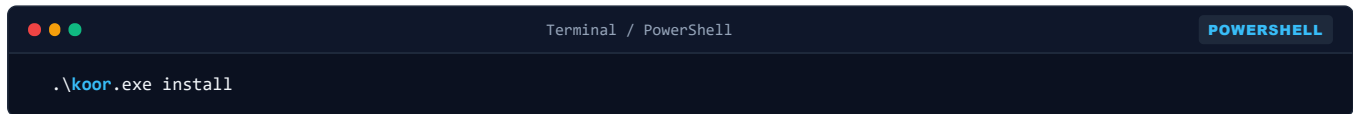
For the most effortless experience, Koor provides a dedicated **One-Click Standalone Windows Setup Wizard** (`KoorSetup.exe`). It requires **zero administrative permissions** and eliminates all manual path configurations:

1. Download `KoorSetup.exe`.
2. Double-click the installer to launch the modern graphical setup wizard.
3. Click **"Rakib Koor (Install Now)"**.
4. The installer automatically:
 - Copies Koor to `%LocalAppData%\Programs\Koor\`.
 - **Automatically adds Koor to your User PATH** in the Windows Registry (`HKCU\Environment`).
 - Broadcasts `WM_SETTINGCHANGE` so all terminals immediately recognize `koor` without restarting your computer.
 - Associates `.koor` files so you can double-click any `.koor` script to run it directly.
 - Adds the **"Ku wad Koor (Run with Koor)"** option to the Windows Explorer right-click context menu.
 - Creates Start Menu and Desktop shortcuts for **Koor REPL**.
5. Once installation finishes, click **"Fur Koor REPL Hadda"** or open any PowerShell / CMD terminal and type `koor` right away!

2.4 Portable Executable & Built-In `koor install`

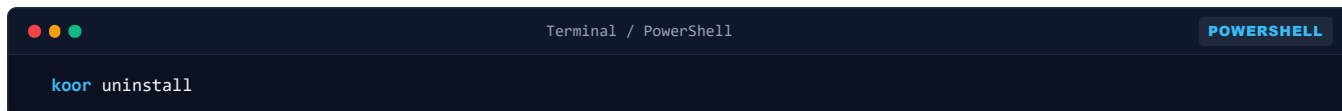
If you downloaded the standalone binary (`koor.exe`) directly without running the setup wizard:

- You can run `koor.exe` immediately from any folder.
- To automatically register it in your Windows PATH and enable double-clicking `.koor` files, simply run:



```
.\koor.exe install
```

- To remove it at any time, run:

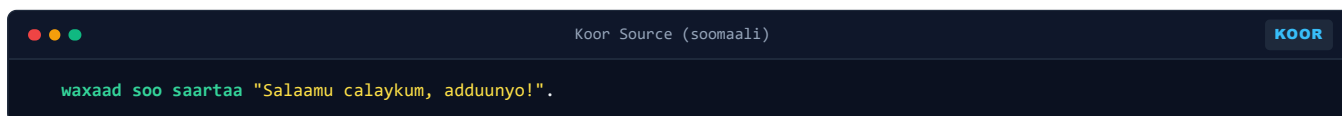
A terminal window with a dark background. The title bar shows three colored dots (red, yellow, green) on the left, 'Terminal / PowerShell' in the center, and a 'POWERSHELL' button on the right. The command 'koor uninstall' is entered in the terminal.

Koor handles all registry modifications, environment notifications, and file associations natively—no manual path editing or computer restarts required!

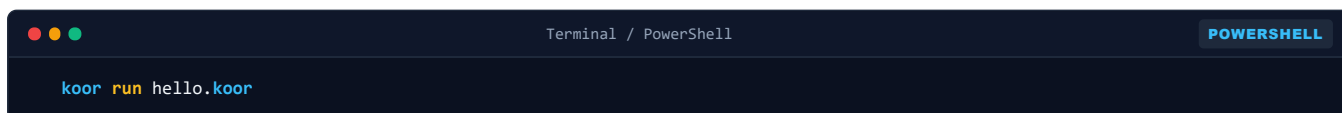
2.5 Setting Up Visual Studio Code & Your First File

Visual Studio Code (VS Code) is the premier free code editor used by professional developers worldwide:

1. Download and install [Visual Studio Code](#).
2. Open VS Code, click **File** $\rightarrow$ **Open Folder...**, and select your Koor directory.
3. In the file explorer, click the **New File** icon and name it `hello.koor`.
4. Type your first line of Koor code:

A Visual Studio Code editor window with a dark theme. The title bar shows three colored dots on the left, 'Koor Source (soomaali)' in the center, and a 'KOOR' button on the right. The code 'waxaad soo saartaa "Salaamu calaykum, adduunyo!".' is typed in the editor.

5. Open the integrated terminal in VS Code by pressing `Ctrl + `` (backtick), and type:

A terminal window with a dark background. The title bar shows three colored dots on the left, 'Terminal / PowerShell' in the center, and a 'POWERSHELL' button on the right. The command 'koor run hello.koor' is entered in the terminal.

(Or press `Ctrl + Shift + B` to trigger the configured one-click run task!)

Complete Terminal Commands

The terminal is the developer's command center. This chapter provides a complete guide to all command-line operations in Koor.

3.1 Demystifying the Command Line for Total Beginners

When beginners first encounter the black terminal screen, it can feel intimidating. Remember:

- In a graphical interface, you interact using the **mouse** by clicking on icons and buttons.
- In the terminal, you interact using **text**. You type a concise instruction, press `Enter`, and the computer immediately obeys.

3.2 Running Koor Files: `koor run`

This is your most frequently used command. It instructs the interpreter to read, parse, and execute a `.koor` source file from top to bottom.

```
Terminal / PowerShell POWERSHELL

# General syntax:
koor run <filename.koor>

# Real examples:
koor run hello.koor
koor run examples_koor\01_guulaysatay.koor
koor run examples_koor\11_ciyaar_qiyaas.koor
```

3.3 The Interactive Playground: `koor repl`

REPL stands for *Read-Eval-Print Loop*. It is an interactive, live scratchpad where you can test ideas instantly without creating a file:

Launch the REPL by typing:

```
Terminal / PowerShell POWERSHELL

koor repl
```

You will see the welcome prompt:

```
Console Output OUTPUT

so Koor Interactive REPL (v1.0.0)
Type your Koor code. Type 'ka bax' or 'exit' to quit.

koor> magac waa "Farxaan".
koor> waxaad soo saartaa "Soo dhawoww {magac}!".
Soo dhawoww Farxaan!
koor> 10 + 25
35
koor> ka bax
```

(To exit the REPL at any time, type `ka bax` or press `Ctrl + C`).

3.4 Validating Syntax: `koor check`

This command scans your `.koor` source file for typos, syntax mistakes, missing quotation marks, or grammar errors **without actually running the program**:

```
Terminal / PowerShell POWERSHELL

koor check examples_koor\06_lixda_tiiir.koor
```

If your code is correct, Koor confirms:

Console Output

OUTPUT

✓ Koodhka naxwihiisu waa sax (Syntax is valid).

3.5 Built-In Help Reference: `koor --help`

Whenever you need a quick reminder of the CLI options, pass the `--help` flag:

Terminal / PowerShell

POWERSHELL

`koor --help`

3.6 Essential Terminal Shortcuts & Tips

Here are pro-tips to speed up your terminal workflow:

1. **The `Tab` Key (Auto-Completion):** Never type out long file paths manually. Type the first few letters (e.g., `koor run ex`) and press `Tab`—the terminal will automatically complete the name!
2. **Up Arrow (`↑`):** Recalls the last command you executed so you don't have to retype it.
3. **`cls` (or `clear`):** Clears cluttered text from your terminal screen.
4. **`Ctrl + C` (Abort):** Immediately halts a running program if it gets stuck in an infinite loop.

Pillar 1 — Variables & Data Types

The first foundational pillar of computer science is **Variables and Data Types**.

Beginner Mental Model (Labeled Storage Boxes):

Think of a variable as a labeled storage box in your computer's memory. When you write `magac waa "Aamina".`, you create a memory box labeled `magac` and place the text `"Aamina"` inside it. Whenever you reference `magac` later in your code, the computer looks inside the box and retrieves `"Aamina"`.

4.1 What is a Variable? The Memory Container Concept

In computer programming, data cannot float around in the computer aimlessly. It must be stored in named memory slots so that you can reuse it, modify it, and calculate with it.

A variable consists of three elements:

1. **The Name (Identifier):** What you call the box (e.g., `x`, `magac`, `da'`).
2. **The Value:** What is stored inside the box (e.g., `12`, `"Aamina"`, `run`).
3. **The Data Type:** The nature of the value (e.g., number, text, boolean).

4.2 Three Ways to Declare Variables (`waa` , `qeex ... inuu yahay` , `=`)

Koor provides three expressive styles to declare and assign variables:

```
1. Natural affirmative statement using 'waa' (recommended):
# 1. Natural affirmative statement using 'waa' (recommended):
magac waa "Aamina".
da' waa 22.

# 2. Descriptive declaration using 'qeex ... inuu yahay' (or 'inay tahay'):
qeex dhibco inuu yahay 95.
qeex xisaab inay tahay 500.

# 3. Standard symbolic assignment:
qiimo = 150.
```

In Somali grammar, `waa` is an equational focus particle that asserts identity or state ("is" or "equals"). Statements naturally end with a period (`.`).

4.3 Variable Identifiers & The Somali Glottal Apostrophe (`'`)

Variable names must adhere to clear rules:

- Must begin with a letter or underscore (`_`).
- Can contain letters, digits, and underscores.
- **Native Apostrophe Support:** In the Somali Latin alphabet, the apostrophe (*galaasad*) is a genuine consonant representing the glottal stop. Koor natively permits the apostrophe inside identifiers:

```
Koor Source (soomaali)
da' waa 25.
go'aan waa "Wanaagsan".
```

- Cannot contain spaces or mathematical operators (e.g., `wadarta guud` is invalid; write `wadarta_guud`).
- Cannot use reserved language keywords like `waa`, `haddii`, `hawl`.

4.4 Variables vs. String Literals (`x` vs. `"x"`)

This is one of the most critical concepts for beginners to understand:

```
Koor Source (soomaali) KOOR

x waa 12.

daabac "x". # Output: x (Prints the literal character 'x')
daabac x.   # Output: 12 (Evaluates the variable x and prints 12)
```

⚠ The Quotation Mark Rule:

* Anything enclosed in quotes (`" ... "` or `' ... '`) is a **String Literal**. The computer treats it as literal characters and prints it verbatim. It will **never** look for a variable named `x` when you write `"x"`.

When you omit quotes (`daabac x`), you tell the computer: "Look into memory, find the box named `x`, and give me what is inside!"*

* If you want to embed a variable inside text, use curly braces: `daabac "Qiimaha waa: {x}"`.

4.5 Natural Verbal In-Place Modifications

Instead of cryptic symbolic operators like `+=`, `-=`, `*=`, and `/=`, Koor lets you express modifications using authentic, spoken Somali sentences. Koor supports two natural grammatical word orders:

Style 1: Subject-First (`<target> waxaad ku dartaa <qiimo>`)

This is the most popular, intuitive syntax where you name the variable first:

```
Koor Source (soomaali) KOOR

da' waa 20

da' waxaad ku dartaa 5      # da' = da' + 5 (Now 25)
da' waxaad ka jartaa 2     # da' = da' - 2 (Now 23)
da' waxaad ku dhufataa 2   # da' = da' * 2 (Now 46)
da' waxaad u qaybisaa 4    # da' = da' / 4 (Now 11.5)

soo saar "Da'da cusub waa: {da'}"
```

Style 2: Action-First (`waxaad <qiimo> ku dartaa <target>`)

Ideal for loop counters and verbal instructions:

```
Koor Source (soomaali) KOOR

tirsade waa 1

waxaad 1 ku dartaa tirsade # tirsade increases by 1 (Now 2)
waxaad 5 ku dartaa tirsade # tirsade increases by 5 (Now 7)
waxaad 2 ka jartaa tirsade # tirsade decreases by 2 (Now 5)
```

You can also use the concise short forms: `ku dar 10 qiimo` or `ka jar 5 qiimo`.

4.6 The 4 Primitive Data Types

Every piece of data in Koor belongs to a specific type:

Type Name	Somali Keyword	Description	Examples
Number	<code>tiro</code>	Integers and decimal floating-point values	<code>42</code> , <code>-15</code> , <code>3.1415</code> , <code>0.5</code>
String	<code>qoraal</code>	Text enclosed in double or single quotes	<code>"Muqdisho"</code> , <code>'Boosaaso'</code>
Boolean	<code>run_been</code>	Binary logical truth values	<code>run</code> (True), <code>been</code> (False)
Null / None	<code>waxba</code>	The explicit absence of a value	<code>waxba</code>

4.7 Type Inspection & Explicit Type Casting

To inspect what type a variable currently holds, use `nooc()`:

```
Koor Source (soomaali) KOOR

waxaad soo saartaa nooc(100).      # "tiro"
waxaad soo saartaa nooc("Koor").   # "qoraal"
waxaad soo saartaa nooc(run).      # "run_been"
waxaad soo saartaa nooc(waxba).    # "waxba"
```

To convert explicitly between types:

```
Koor Source (soomaali) KOOR

# String to Number:
x waa tiro("45").      # Converts "45" string into number 45

# Number to String:
hadal waa qoraal(100).  # Converts number 100 into string "100"

# Number to Boolean:
xaalad waa run_been(1). # Converts 1 into run (True)
```

Pillar 2 — Operators & Expressions

Operators are the tools we use to calculate values, transform data, and make comparisons.

Beginner Mental Model (The Balancing Scale):

Think of operators as balancing scales or a kitchen calculator. They take two inputs (such as `10` and `5`) and evaluate a result—either performing math (`10 + 5` $\rightarrow$ `15`) or checking a truth condition (`10 > 5` $\rightarrow$ `run`).

5.1 Arithmetic Operators & Natural Zero-Signs Math

Koor supports two expressive styles to perform calculations:

Style 1: Natural Zero-Signs Math (Pure Spoken Somali)

Koor allows you to write natural, readable mathematical operations without typing mathematical symbols:

```
Koor Source (soomaali) KOOR

k1 waa 15 ku dar 10      # 25  (Addition: ku dar = +)
k2 waa 30 ka jar 12      # 18  (Subtraction: ka jar = -)
k3 waa 7 ku dhufo 8      # 56  (Multiplication: ku dhufo = *)
k4 waa 100 u qaybi 4      # 25.0 (Division: u qaybi = /)
k5 waa 17 haraaga 5      # 2   (Modulo/Remainder: haraaga = %)
```

Style 2: Classic Mathematical Operators

Standard algebraic signs are also supported with full operator precedence:

```
Koor Source (soomaali) KOOR

wadarta waa 15 + 10.    # 25  (+)
farqiga waa 30 - 12.    # 18  (-)
taranka waa 7 * 8.      # 56  (*)
qaybta waa 100 / 4.     # 25.0 (/)
hadhaaga waa 17 % 5.    # 2   (%)
```

Operator Precedence (Order of Operations):

Multiplication (`*`), division (`/`), and modulo (`%`) are calculated **before** addition (`+`) and subtraction (`-`). Use parentheses (`...`) to force calculations to occur first:

```
``soomaali
natiijo1 waa 5 + 3 * 2. # 5 + 6 = 11
natiijo2 waa (5 + 3) 2. # 8 2 = 16
````
```

### 5.2 Natural Somali Comparison Phrases vs. Symbolic Operators

Comparisons evaluate whether a relationship holds between two values, producing `run` (True) or `been` (False). Koor natively supports **both** authentic Somali verbal phrases and standard mathematical symbols:

| Natural Somali Phrase                   | Symbol            | Example                               | Meaning                  |
|-----------------------------------------|-------------------|---------------------------------------|--------------------------|
| ay la mid tahay / uu la mid yahay       | <code>==</code>   | <code>x ay la mid tahay 10</code>     | Is equal to              |
| aysan la mid ahayn / uusan la mid ahayn | <code>≠</code>    | <code>x aysan la mid ahayn 0</code>   | Is not equal to          |
| uu ka weyn yahay / ay ka weyn tahay     | <code>&gt;</code> | <code>da' ay ka weyn tahay 18</code>  | Greater than             |
| uu ka yar yahay / ay ka yar tahay       | <code>&lt;</code> | <code>qiimo uu ka yar yahay 50</code> | Less than                |
| ka weyn yahay ama la mid yahay          | <code>≥</code>    | <code>dhibco ≥ 80</code>              | Greater than or equal to |
| ka yar yahay ama la mid tahay           | <code>≤</code>    | <code>i ≤ 5</code>                    | Less than or equal to    |

Both styles are completely interchangeable:

```

Koor Source (soomaali) KOOR

x waa 20.

waxaad soo saartaa x > 15. # run
waxaad soo saartaa x ay la mid tahay 20. # run
waxaad soo saartaa x uu ka yar yahay 10. # been

```

### 5.3 Gender Agreement in Comparisons ( `uu` vs. `ay` )

Somali grammar uses grammatical gender for pronouns:

- `uu` is masculine (e.g., `qiimo uu ka weyn yahay 50`).
- `ay` is feminine (e.g., `da' ay ka weyn tahay 18`).

In Koor, the interpreter gracefully accepts both forms! Whether you write `uu ka weyn yahay` or `ay ka weyn tahay`, the logic evaluates accurately.

### 5.4 Logical Operators ( `sidoo kale`, `iyo`, `ama`, `ma`, `ma aha` )

Logical operators allow combining multiple conditions:

- Logical AND** ( `sidoo kale` / `iyo` ): Evaluates to `run` only if **both** sides are true.

```

Koor Source (soomaali) KOOR

haddii da' >= 18 sidoo kale shatiga ay la mid tahay run:
 waxaad soo saartaa "Waad wadi kartaa gaariga.".

```

- Logical OR** ( `ama` ): Evaluates to `run` if **at least one** side is true.

```

Koor Source (soomaali) KOOR

haddii maalintu ay la mid tahay "Jimce" ama maalintu ay la mid tahay "Sabti":
 waxaad soo saartaa "Waa fasax!".

```

- Logical NOT** ( `ma` / `ma aha` ): Inverts the truth value ( `run` becomes `been`, `been` becomes `run` ).

```

Koor Source (soomaali) KOOR

haddii ma roob:
 waxaad soo saartaa "Bannaanka u bax.".

```

## Pillar 3 — Input and Output (I/O)

Programs must interact with human beings to be truly useful.

### Beginner Mental Model (The Billboard and Microphone):

- **Output** ( `waxaad soo saartaa` ): The program's billboard, displaying messages on the screen.
- **Input** ( `waydiin()` ): The program's microphone, listening for whatever the user types on the keyboard.

#### 6.1 Output Commands: `waxaad soo saartaa`, `daabac`, `qor`, `soo saar`

Koor provides several synonyms for printing to the terminal:

```
Koor Source (soomaali) KOOR

waxaad soo saartaa "Ku soo dhawoow Koor!". # Formal natural phrase
daabac "Qoraal degdeg ah.". # Short form
qor "Qoraal kale.". # Short form
soo saar "Hab kale oo loo qoro.". # Short form
```

#### 6.2 Printing Variables, Literals, and Comma-Separated Values

You can print variables directly or print multiple values separated by commas:

```
Koor Source (soomaali) KOOR

magac waa "Cali".
dhibco waa 95.

Printing variables directly:
daabac magac. # Output: Cali

Printing multiple comma-separated values:
daabac "Ardayga:", magac, "Dhibcaha:", dhibco.
Output: Ardayga: Cali Dhibcaha: 95
```

#### 6.3 Dynamic String Interpolation ( `{variable}` and `{object.field}` )

Instead of concatenating strings with `+`, embed variables inside text using curly braces `{ ... }`:

```
Koor Source (soomaali) KOOR

magaalo waa "Boosaaso".
heerkul waa 32.

waxaad soo saartaa "Magaalada {magaalo} heerkulkeedu waa {heerkul}°C.".
Output: Magaalada Boosaaso heerkulkeedu waa 32°C.
```

Interpolation also supports accessing properties of dictionaries:

```
Koor Source (soomaali) KOOR

qof waa {"magac": "Aamina", "shaqo": "Dhaqtar"}.
waxaad soo saartaa "{qof.magac} waa {qof.shaqo}.".
Output: Aamina waa Dhaqtar.
```

#### 6.4 Interactive Keyboard Input with `waydiin()`

To prompt the user for keyboard input, use `waydiin()` (or its aliases `weydiin`, `waxaad waydiisaa`):



```
magac waa waydiin("Fadlan qor magacaaga: ").
waxaad soo saartaa "Salaan sare, {magac}!".
```

## 6.5 Reading Numbers: Why `tiro(waydiin(...))` is Mandatory

### ⚠ Critical Beginner Rule:

The `waydiin()` function **always** returns the user's input as text ( `qoraal` ), even if the user types digits.

If the user types `20`:

```
``soomaali
```

```
sanad_qoraal waa waydiin("Geli tirada: "). # Returns "20" (text)
```

```
If you do sanad_qoraal + 5, it throws an error or concatenates text!
```

```
``
```

Whenever you expect a number for math calculations, you **must** wrap `waydiin()` inside `tiro(...)`:

```
``soomaali
```

```
da' waa tiro(waydiin("Geli da'daada: ")). # Converts to number 20
```

```
sanad_dambe waa da' + 1. # 21 (correct math!)
```

```
waxaad soo saartaa "Sannadka dambe waxaad noqonaysaa: {sanad_dambe}.".
```

```
``
```

## Pillar 4 — Control Flow (Decisions & Loops)

Control flow structures dictate which branches of code execute and how many times statements repeat.

### **Beginner Mental Model (Road Forks and Conveyor Belts):**

Normally, code runs sequentially from top to bottom. Control flow gives you steering power:

- **Decisions ( `haddii` )**: Like a fork in the road with traffic signals, the computer chooses which path to take based on current conditions.
- **Loops ( `ku celi` / `inta` )**: Like an automated factory conveyor belt, the computer repeats a task multiple times without requiring you to duplicate code.

### 7.1 Indentation Rules, Colons ( `:` ), and Code Blocks

In Koor, blocks of code that belong inside a condition, loop, or function are defined by **Indentation** (leading whitespace of 4 spaces or one Tab) following a colon ( `:` ):

```
haddii dhibco >= 50:
 # Everything indented 4 spaces belongs inside this block!
 waxaad soo saartaa "Waad gudubtay.".
 waxaad soo saartaa "Hambalyo!".

This unindented line runs after the block finishes:
waxaad soo saartaa "Dhammaad.".
```

### 7.2 Single-Line Spoken Conditionals

For simple, one-line decisions, Koor supports natural spoken sentences without colons or line breaks:

```
x waa 12.
haddi x ay la mid tahay 12 waxaad soo saartaa "Guul, x waa 12!".
```

### 7.3 Multi-Branch Conditionals ( `haddii` , `hadii kale` , `kale` )

For multi-condition branching, use `haddii` , followed by `hadii kale oo ay` or `hadii kale oo uu` for "else if" branches, and `kale:` (or `haddii kale:` ) as the fallback default:

```
dhibco waa 84.

haddii dhibco >= 90:
 waxaad soo saartaa "Darajo: A (Heer Sare)".
hadii kale oo ay dhibco >= 80:
 waxaad soo saartaa "Darajo: B (Aad u Wanaagsan)".
hadii kale oo uu dhibco >= 60:
 waxaad soo saartaa "Darajo: C (Dhexdhexaad)".
kale:
 waxaad soo saartaa "Darajo: D (Dhacay)".
```

### 7.4 Truthiness Rules in Koor (Truthy vs. Falsy Values)

When evaluating expressions inside `haddii` or `inta` , Koor treats the following values as **Falsy**:

- `been` (Boolean False)
- `waxba` (Null / None)
- `0` (Numeric zero)

- `""` (Empty string)

All other values—including non-zero numbers, non-empty strings, and populated lists—are considered **Truthy**.

### 7.5 Fixed-Count Loops ( `ku celi ... jeer:` )

When you want to repeat an exact block of code a known number of times:

```
Koor Source (soomaali) KOOR

ku celi 4 jeer:
 waxaad soo saartaa "Koor waa luuqad fudud oo awood badan."
```

### 7.6 Conditional While Loops ( `inta` )

Executes repeatedly as long as a condition remains true. Notice that in authentic Koor, colons ( `:` ) are completely optional:

```
Koor Source (soomaali) KOOR

soo saar "--- Wareegga 'inta' ---"
tirsade waa 1

inta tirsade ka yar yahay ama la mid yahay 3
 soo saar "Tirsade waa: {tirsade}"
 waxaad 1 ku dartaa tirsade
```

In each step through the loop:

1. `tirsade` starts at 1, prints `Tirsade waa: 1`, and increments to 2 via `waxaad 1 ku dartaa tirsade`.
2. `tirsade` is 2, prints `Tirsade waa: 2`, and increments to 3.
3. `tirsade` is 3, prints `Tirsade waa: 3`, and increments to 4.
4. When `tirsade` reaches 4, `tirsade ka yar yahay ama la mid yahay 3` evaluates to `been` (False), and the loop terminates!

### 7.7 Collection Iteration ( `mid kastoo ku jira` )

Koor provides natural, spoken Somali expressions to iterate through every element in a list:

```
Koor Source (soomaali) KOOR

ardayda waa liiska "Cali" iyo "Aamina" iyo "Warsame"

Style A: Inferred Collection (iterates directly over the active list):
mid kastoo ku jira (qof)
 soo saar "Ku soo dhawow {qof}!"

Style B: Explicit Collection with parenthesized loop variable:
mid kastoo ku jira ardayda (qof)
 soo saar "Ku soo dhawow {qof}!"

Style C: Pure Zero-Signs Trailing Variable:
mid kastoo ku jira ardayda qof
 soo saar "Ku soo dhawow {qof}!"
```

### 7.8 Loop Jump Controls: Break ( `ka bax` ) and Continue ( `ka bood` )

- `ka bax.` — **Break:** Immediately exits and terminates the enclosing loop.
- `ka bood.` — **Continue:** Skips the remainder of the current iteration and jumps directly to the next.



```
i waa 0.
inta i < 8:
 waxaad ku dartaa 1 i.
 haddii i ay la mid tahay 3:
 ka bood. # Skips printing 3
 haddii i ay la mid tahay 6:
 ka bax. # Terminates loop when i reaches 6
 waxaad soo saartaa i.
```

## Pillar 5 — Functions (Hawlaha)

Functions allow you to write a block of code once, give it a name, and reuse it whenever needed.

### **Beginner Mental Model (The Specialized Assistant):**

Think of a function as a specialized recipe or a hired assistant. Instead of repeating 10 lines of complex calculation every time, you define a named function ( `hawl` ). Whenever you call its name and provide the ingredients (arguments), it performs the task and hands back the result!

#### 8.1 Defining Block Functions ( `hawl` )

Declare a function using `hawl`, followed by the function name, its parameter list inside parentheses, a colon ( `:` ), and an indented body:

```
hawl salaam(magac):
 waxaad soo saartaa "Salaamu Calaykum, {magac}!".

Invoking the function:
salaam("Warsame").
salaam("Hodan").
```

#### 8.2 Returning Values ( `waxaad soo celisaa` ) vs. Printing ( `daabac` )

Beginners often confuse **printing** with **returning**:

- `daabac` / `waxaad soo saartaa`: Displays text on the terminal screen for human eyes. The computer cannot use that output in further calculations.
- `waxaad soo celisaa` (or `celi`): Hands the calculated result back to your code so you can save it into a variable or use it in other math!

```
hawl xisaabi_bedka(ballac, dherer):
 bed waa ballac * dherer.
 waxaad soo celisaa bed.

natiijo waa xisaabi_bedka(6, 7).
waxaad soo saartaa "Bedka goobtu waa: {natiijo} mitir oo labajibbaaran.".
Output: Bedka goobtu waa: 42 mitir oo labajibbaaran.
```

#### 8.3 Structured Declarative Functions ( `magaceed waa` , `tibxuhu waa` , `kaydi natiijada` )

Koor introduces a structured, declarative syntax that expresses function definitions as clear, natural Somali blocks:

```
hawl
magaceed waa : isku dhufasho
tibxuhu waa : x , y
hawshu waa : x ku dhufo y
kaydi natiijada
```

Every structured function is organized around four clear directives:

- `magaceed waa`: Assigns the unique name to the function.
- `tibxuhu waa`: Declares the parameters (variables) the function accepts.
- `hawshu waa`: Specifies the calculation, algorithm, or code statements to execute.
- `kaydi natiijada`: Flags that the result should be preserved and returned to the caller.

If a function does not need parameters (a zero-argument function), you simply omit `tibxuhu waa` :

```
Koor Source (soomaali) KOOR

hawl
magaceed waa : salaam
hawshu waa : soo saar "Ku soo dhawow dunida Koor!"
```

#### 8.4 Function Invocation with `qabo hawshan`

The primary and most idiomatic way to invoke functions in Koor is using the phrase `qabo hawshan` :

```
1. Calling a function with arguments (function name without quotes): KOOR

1. Calling a function with arguments (function name without quotes):
natiijo waa qabo hawshan (isku dhufasho) (6, 7)
soo saar "Natiijadu waa: {natiijo}" # 42

2. Calling a zero-argument function (no second brackets needed):
qabo hawshan (salaam)
```

##### 💡 Key Rules for `qabo hawshan` :

1. **No Quotation Marks:** Place the function name directly inside parentheses without quotes: `(isku dhufasho)` .
2. **Optional Second Brackets:** If the function takes parameters, pass the arguments in the second set of parentheses: `(6, 7)` or `(x, y)` .  
If the function has no parameters, **do not include second brackets**: `qabo hawshan (salaam)` .
3. **Assignment:** You can store the returned value directly into a variable using `waa :` `natiijo waa qabo hawshan (isku dhufasho) (x, y)` .

#### Complete Interactive Program Example:

Here is a complete, working Koor program bringing together keyboard input, natural arithmetic, structured function definition, and invocation with `qabo hawshan` :

```
1. Capture user inputs from terminal KOOR

1. Capture user inputs from terminal
x waa tiro waydiin gali lambarka koobaad
y waa tiro waydiin gali lambarka labaad

2. Define the structured function
hawl
magaceed waa : isku dhufasho
tibxuhu waa : x , y
hawshu waa : x ku dhufo y
kaydi natiijada

3. Invoke the function using qabo hawshan
natiijo waa qabo hawshan (isku dhufasho) (x, y)

4. Display the computed result
soo saar "tiradaadu waa {natiijo}"
```

#### 8.5 Parameters, Arguments, and Local Scope

- **Parameters:** The variable names defined in the function declaration (e.g., `x` and `y` ).
- **Arguments:** The actual values you pass in when calling the function (e.g., `6` and `7` ).
- **Local Scope:** Variables created inside a function exist **only** inside that function. Once the function finishes running, those local variables are removed from memory.

#### 8.6 Recursive Functions (Functions Calling Themselves)

A function can call itself to solve nested problems, provided it has a base case to stop recursion:



```
hawl faktooriyal(n):
 haddii n <= 1:
 waxaad soo celisaa 1.
 waxaad soo celisaa n * faktooriyal(n - 1).

waxaad soo saartaa faktooriyal(5). # 120
```

## Pillar 6 — Data Structures

Data structures allow organizing, grouping, and manipulating collections of data in memory.

### Beginner Mental Model (Shopping Lists and ID Cards):

- **List ( `liis` )**: Think of a grocery shopping list. It is an ordered sequence of items accessed by numerical position (0, 1, 2, ...).
- **Dictionary ( `qaamuus` )**: Think of an identity card or passport. It contains labeled fields paired with their values: `"magac": "Caḷi"`, `"da": 25`.

### 9.1 Ordered Lists ( `liis` ): Indexing, Zero-Based Access, & Mutation

Lists are ordered collections enclosed in square brackets `[ ... ]`. Items are accessed by **index**, starting at 0:

```

ardayda waa ["Caḷi", "Faadumo", "Warsame", "Hodan"].

Reading by index:
waxaad soo saartaa ardayda[0]. # "Caḷi" (First item)
waxaad soo saartaa ardayda[2]. # "Warsame" (Third item)

Mutating an element in place:
ardayda[1] = "Deeqa".
waxaad soo saartaa ardayda. # ["Caḷi", "Deeqa", "Warsame", "Hodan"]

```

### 9.2 Built-In List Methods

Koor lists provide built-in mutation methods:

- `.ku_dar(item)` — Appends an item to the end of the list.
- `.ka_saar()` — Removes and returns the last item.
- `.kala_sooc()` — Sorts items in ascending numerical/alphabetical order.
- `.rog()` — Reverses the list in place.
- `dherer(list)` — Returns the total count of items.

```

tirooyin waa [40, 10, 30, 20].

tirooyin.ku_dar(50). # [40, 10, 30, 20, 50]
tirooyin.kala_sooc(). # [10, 20, 30, 40, 50]
waxaad soo saartaa tirooyin.
waxaad soo saartaa "Tirada xubnaha: " + qoraal(dherer(tirooyin)). # 5

```

### 9.3 Key-Value Dictionaries ( `qaamuus` ): Bracket & Dot Notations

Dictionaries store data as key-value pairs inside curly braces `{ ... }`:

```
Koor Source (soomaali) KOOR

qof waa {
 "magac": "Axmed",
 "da": 28,
 "shaqo": "Injineer",
 "magaalo": "Kismaayo"
}.

Accessing values (both bracket and dot notations work!):
waxaad soo saartaa qof["magac"]. # "Axmed"
waxaad soo saartaa qof.shaqo. # "Injineer"

Updating and adding key-value pairs:
qof["da"] = 29.
qof.khibrad = "5 sano".

waxaad soo saartaa "{qof.magac} waa {qof.shaqo} ku sugan {qof.magaalo}."
```

You can even modify dictionary values using natural verbal statements:

```
Koor Source (soomaali) KOOR

akoon waa {"baaqi": 200}.
waxaad ku dartaa 50 akoon["baaqi"]. # akoon["baaqi"] is now 250!
```

## 9.4 Text Sequences & String Methods

Text strings in Koor can be transformed using chaining methods:

- `.jar()` — Strips whitespace from both ends.
- `.weyneey()` — Converts text to uppercase.
- `.yaree()` — Converts text to lowercase.
- `.beddel(old, new)` — Replaces occurrences of a substring.
- `.kala_bax(delimiter)` — Splits text into a list of words.
- `dherer(text)` — Returns character count.

```
Koor Source (soomaali) KOOR

jumlad waa " soomaaliya ha noolaato ".

waxaad soo saartaa jumlad.jar().weyneey().
Output: SOOMALIYA HA NOOLAATO

ereyo waa jumlad.jar().kala_bax(" ").
waxaad soo saartaa ereyo.
Output: ["soomaaliya", "ha", "noolaato"]
```

## 9.5 Built-In Mathematical & Utility Functions

Koor provides built-in utilities in the global environment:

- `nasiib(min, max)` — Generates a random integer between `min` and `max`.
- `xidid(n)` — Computes the square root of `n`.
- `qaanso(start, stop)` — Generates a range list of numbers.

```
Koor Source (soomaali) KOOR

tirsade_nasiib waa nasiib(1, 10). # Random number from 1 to 10
root_16 waa xidid(16). # 4.0
```

## Real-World Projects (Mashaariic Dhab Ah)

Congratulations! You have now mastered the six core pillars of programming. Now let's apply everything you have learned to build **four complete, practical real-world applications**. Each project includes full executable source code, line-by-line explanations, and realistic terminal outputs.

### 10.1 Project 1: Retail Store POS & Invoice Generator

This project simulates a retail Point of Sale (POS) system for a Somali grocery market. It processes a customer's shopping cart, computes item subtotals, tallies the order total, applies a 10% discount if the order reaches or exceeds \$80, and prints an itemized invoice.

```
dukaan_qaansheeg.koor - Retail POS & Invoice Generator KOOR

dukaan_qaansheeg.koor - Retail POS & Invoice Generator
alaabta waa [
 {"magac": "Bariis 25kg", "qiimo": 24, "xaddi": 2},
 {"magac": "Saliid 5L", "qiimo": 12, "xaddi": 1},
 {"magac": "Sonkor 10kg", "qiimo": 15, "xaddi": 3}
].

wadarta_guud waa 0.

waxaad soo saartaa "=====".
waxaad soo saartaa " 🛒 QAANSHEEGTA DUKAANKA BARAKO ".
waxaad soo saartaa "=====".

mid kastoo ku jira alaabta (shay):
 qiimaha_shayga waa shay["qiimo"] * shay["xaddi"].
 waxaad ku dartaa qiimaha_shayga wadarta_guud.
 waxaad soo saartaa "{shay.magac} x{shay.xaddi} = ${qiimaha_shayga}".

waxaad soo saartaa "-----".
waxaad soo saartaa "Wadarta Hore: ${wadarta_guud}".

qiimo_dhimis waa 0.
haddii wadarta_guud >= 80:
 qiimo_dhimis waa wadarta_guud * 0.10.
 waxaad soo saartaa "Qiimo-dhimis (10%): -${qiimo_dhimis}".
kale:
 waxaad soo saartaa "Qiimo-dhimis: $0".

lacagta_bixinta waa wadarta_guud - qiimo_dhimis.
waxaad soo saartaa "Lacagta Guud ee Bixinta: ${lacagta_bixinta}".
waxaad soo saartaa "=====".
waxaad soo saartaa "Mahadsanid! Mar kale soo dhawow!".
```

#### Detailed Step-by-Step Code Explanation:

- The Cart Data Structure ( `alaabta` ):** We define a list `[ ... ]` containing dictionary records `{ ... }`. Each item contains three properties: item description ( `"magac"` ), unit price ( `"qiimo"` ), and purchase quantity ( `"xaddi"` ).
- Total Accumulator ( `wadarta_guud = 0` ):** We initialize a numerical variable at zero to accumulate the running grand total.
- The Item Processing Loop ( `mid kastoo ku jira alaabta ( shay ):` ):** This loop iterates through every grocery item. For each item:
  - It calculates the line item subtotal: `qiimaha_shayga waa shay["qiimo"] * shay["xaddi"].`
  - It increments the running grand total using Koor's natural verbal modification: `waxaad ku dartaa qiimaha_shayga wadarta_guud.`
  - It prints the formatted receipt line: `{shay.magac} x{shay.xaddi} = ${qiimaha_shayga}.`
- Conditional Discount Logic ( `haddii wadarta_guud >= 80:` ):** We check if the customer spent \$80 or more. If true, the system applies a 10% discount ( `wadarta_guud * 0.10` ); otherwise, discount remains \$0.

5. **Final Net Payable** ( `lacagta_bixinta` ): We subtract the discount from the initial subtotal and print the receipt footer.

**Output / Result:**

```
=====
 QAANSHEEGTA DUKAANKA BARAKO
=====
Bariis 25kg x2 = $48
Saliid 5L x1 = $12
Sonkor 10kg x3 = $45

Wadarta Hore: $105
Qiimo-dhimis (10%): -$10.5
Lacagta Guud ee Bixinta: $94.5
=====
Mahadsanid! Mar kale soo dhawoow!
```

## 10.2 Project 2: Student Grading & Class Analytics System

This project models an academic grading management system. It translates student examination scores into formal letter grades ( `A` to `D` ), tallies class totals, calculates the average class performance, and generates a formatted academic report.

```
arday_darajo.koor - Student Grading & Class Analytics

arday_darajo.koor - Student Grading & Class Analytics
ardayda waa [
 {"magac": "Cali Axmed", "dhibco": 94},
 {"magac": "Faadumo Jaamac", "dhibco": 82},
 {"magac": "Warsame Cali", "dhibco": 68},
 {"magac": "Hodan Nuur", "dhibco": 45}
].

hawl soo_saar_darajo(dhibco):
 haddii dhibco >= 90:
 waxaad soo celisaa "A (Heer Sare)".
 hadii kale oo ay dhibco >= 80:
 waxaad soo celisaa "B (Wanaagsan)".
 hadii kale oo ay dhibco >= 60:
 waxaad soo celisaa "C (Dhexdhexaad)".
 kale:
 waxaad soo celisaa "D (Dhacay)".

waxaad soo saartaa "===== ".
waxaad soo saartaa " NATIIJADA IMTIXAANKA ARDAYDA ".
waxaad soo saartaa "===== ".

wadarta_dhibcaha waa 0.

mid kastoo ku jira ardayda (arday):
 darajo waa soo_saar_darajo(arday["dhibco"]).
 waxaad ku dartaa arday["dhibco"] wadarta_dhibcaha.
 waxaad soo saartaa "{arday.magac}: {arday.dhibco} dhibcood -> {darajo}".

tirada_ardayda waa dherer(ardayda).
celcelis waa wadarta_dhibcaha / tirada_ardayda.

waxaad soo saartaa "----- ".
waxaad soo saartaa "Celceliska Fasalka: {celcelis} dhibcood".
waxaad soo saartaa "Wadarta Ardayda: {tirada_ardayda} arday".
waxaad soo saartaa "===== ".
```

#### Detailed Step-by-Step Code Explanation:

1. **The Grading Function** ( `haw1 soo_saar_darajo(dhibco)` ): A modular function that encapsulates the grading rubric. It takes a numerical score and uses `haddii`, `hadii kale oo ay`, and `kale` to evaluate the corresponding tier, returning a descriptive letter grade via `waxaad soo celisaa`.
2. **Student Records** ( `ardayda` ): An array of dictionaries storing student profiles with their respective exam scores.
3. **Iterative Report Generation**: The `mid kastoo` loop invokes `soo_saar_darajo()` for each student, adds their score to `wadarta_dhibcaha`, and prints their formatted report card line.
4. **Statistical Class Average** ( `celcelis` ): We use the built-in function `dherer(ardayda)` to count total students dynamically, then divide total marks by the student count to compute the class average score.

#### Output / Result:

```

=====
NATIIJADA IMTIXAANKA ARDAYDA
=====
Cali Axmed: 94 dhibcood -> A (Heer Sare)
Faadumo Jaamac: 82 dhibcood -> B (Wanaagsan)
Warsame Cali: 68 dhibcood -> C (Dhexdhexaad)
Hodan Nuur: 45 dhibcood -> D (Dhacay)

Celceliska Fasalka: 72.25 dhibcood
Wadarta Ardayda: 4 arday
=====

```

### 10.3 Project 3: Mobile Money & Digital Banking Wallet (EVC Plus / Zaad)

This project simulates a digital mobile money wallet (resembling EVC Plus, Sahal, or Zaad service). It implements account creation, money deposits, withdrawals with strict insufficient balance checks, and balance inquiry statements.

```

banki.koor — Mobile Money & Bank Account Manager
banki.koor — Mobile Money & Bank Account Manager
haw1 abuur_akoon(magac, baaqi_bilow):
 waxaad soo celisaa {
 "magac": magac,
 "baaqi": baaqi_bilow
 }.

haw1 dhigo_lacag(akoon, xaddi):
 waxaad ku dartaa xaddi akoon["baaqi"].
 waxaad soo saartaa "Waxaad dhigatay ${xaddi}. Baaqiga cusub waa: ${akoon.baaqi}.".

haw1 la_bax_lacag(akoon, xaddi):
 haddii xaddi > akoon["baaqi"]:
 waxaad soo saartaa "❌ Khalad: Lacag kugu filan kuma jirto akoonka!".
 kale:
 waxaad ka jartaa xaddi akoon["baaqi"].
 waxaad soo saartaa "Waxaad la baxday ${xaddi}. Baaqiga ku haray: ${akoon.baaqi}.".

haw1 daabac_warbixin(akoon):
 waxaad soo saartaa "===== ".
 waxaad soo saartaa "Akoonka: {akoon.magac} | Baaqiga: ${akoon.baaqi}.".
 waxaad soo saartaa "===== ".

--- Practical Test Scenario ---
akoon1 waa abuur_akoon("Deeqa Axmed", 500).

dhigo_lacag(akoon1, 200). # Balance: $700
la_bax_lacag(akoon1, 150). # Balance: $550
la_bax_lacag(akoon1, 800). # ❌ Error: Insufficient funds
daabac_warbixin(akoon1).

```

#### Detailed Step-by-Step Code Explanation:

1. **Account State Factory** ( `abuur_akoon` ): Generates and returns a state dictionary containing the account owner's name and opening balance ( `"baaqi"` ).
2. **Deposit Operation** ( `dhigo_lacag` ): Takes the account object and an amount, mutating the internal balance using natural Somali verbal addition: `waxaad ku dartaa xaddi akoon["baaqi"]`.
3. **Guarded Withdrawal** ( `la_bax_lacag` ): Validates the requested transaction against current balance before deducting funds: `haddii xaddi > akoon["baaqi"]`. If funds are insufficient, it rejects the transaction with an error notice. If funds are sufficient, it deducts the amount via `waxaad ka jartaa` and prints the updated balance.
4. **Statement Generation** ( `daabac_warbixin` ): Displays the current account summary.

**Output / Result:**

```
Waxaad dhigatay $200. Baaqiga cusub waa: $700.
Waxaad la baxday $150. Baaqiga ku haray: $550.
❌ Khalad: Lacag kugu filan kuma jirto akoonka!
=====
Akoonka: Deeqa Axmed | Baaqiga: $550
=====
```

#### 10.4 Project 4: Interactive Number Guessing Game (With Hints & Attempts)

This project builds a complete, interactive terminal game. The computer picks a secret random number between 1 and 50 using `nasiib(1, 50)`. The player has 5 attempts to guess it. After each guess, the game gives real-time feedback whether the guess was "too high" or "too low," while keeping track of all previous guesses.

```
ciyaar_qiyaas.koor - Interactive Number Guessing Game KOOR

ciyaar_qiyaas.koor - Interactive Number Guessing Game
waxaad soo saartaa "=====".
waxaad soo saartaa " 🎮 KU SOO DHAWOOW CIYAARTA QIYAASTA TIRADA! ".
waxaad soo saartaa "=====".

magac waa waydiin("Fadlan geli magacaaga, saaxiib: ").
waxaad soo saartaa "Soo dhawoow {magac}! Waxaan doortay tiro u dhaxaysa 1 iyo 50.".
waxaad soo saartaa "Waxaad haysataa 5 fursadood oo aad ku qiyaasto.\n".

Generate a secret random number (1 to 50)
tirada_qarsoon waa nasiib(1, 50).

Game tracking state
isku_dayo waa 0.
fursado_haray waa 5.
waad_badisay waa been.
qiyaasaha_hore waa [].

inta fursado_haray uu ka weyn yahay 0:
 waxaad soo saartaa "-----".
 waxaad soo saartaa "Fursadaha kuu haray: {fursado_haray} | Isku-day: {isku_dayo}.".

 qiyaas_qoraal waa waydiin("Geli tiradaada aad qiyaastay (1-50): ").
 qiyaas waa tiro(qiyaas_qoraal).

 waxaad ku dartaa 1 isku_dayo.
 waxaad ka jartaa 1 fursado_haray.
 qiyaasaha_hore.ku_dar(qiyaas).

Check the guess
haddii qiyaas ay la mid tahay tirada_qarsoon:
 waad_badisay waa run.
 waxaad soo saartaa "\n 🎉 HAMBALYO {magac}! WAAD GUULAYSATAY!".
 waxaad soo saartaa "Tirada saxda ahayd waa: {tirada_qarsoon}.".
 waxaad soo saartaa "Waxaad ku heshay {isku_dayo} isku-day oo keliya!".
 ka bax.
hadii kale oo uu qiyaas uu ka weyn yahay tirada_qarsoon:
 waxaad soo saartaa " 📧 Aad bay u WEYN TAHAY! Tiri tiro ka yar.".
hadii kale oo uu qiyaas uu ka yar yahay tirada_qarsoon:
 waxaad soo saartaa " 📧 Aad bay u YAR TAHAY! Tiri tiro ka weyn.".

If attempts are exhausted without finding the secret number
haddii ma waad_badisay:
 waxaad soo saartaa "\n=====".
 waxaad soo saartaa " 😞 WAAN KA XUMAHAY {magac}, FURASADIHII WAY DHAMMAADEEN!".
 waxaad soo saartaa "Tirada saxda ah ee qarsoonayd waxay ahayd: {tirada_qarsoon}.".
 waxaad soo saartaa "Qiyaasihii aad samaysay waxay ahaayeen: " + qoraal(qiyaasaha_hore).
 waxaad soo saartaa "Mar kale isku day! Nabad gelyo!".
```

#### #### Detailed Step-by-Step Code Explanation:

- 1. Random Generation ( `nasiib(1, 50)` ):** The built-in `nasiib` function produces an unpredictable pseudo-random integer between 1 and 50 inclusive.
- 2. State Management:**
  - `isku_dayo` : Tracks the number of guesses attempted.
  - `fursado_haray` : Initialized at 5 and decremented after each turn.
  - `waad_badisay` : A boolean flag initialized to `been` (False).
  - `qiyaasaha_hore` : A list storing past attempts via `.ku_dar(qiyaas)`.
- 3. The Interactive Game Loop ( `inta fursado_haray > 0:` ):** The while loop controls gameplay as long as the user has remaining attempts.

4. **Reading and Converting Input:** `waydiin()` captures player text input, and `tiro()` immediately converts it to a number for numerical comparison.

5. **Hint Branching & Victory Breakout:**

- If `qiyaas == tirada qarsoon`, the player wins: `waad_badisay` becomes `run`, celebratory messages are output, and `ka bax.` terminates the loop immediately.
- If too high or too low, clear navigational hints are printed.

6. **Game Over Condition ( `haddii ma waad_badisay:` ):** If the loop ends without victory, the game reveals the secret number and displays the player's history of guesses.

**Sample Terminal Run:**

```
=====
🎮 KU SOO DHAWOOW CIYAARTA QIYAASTA TIRADA!
=====
Fadlan geli magacaaga, saaxiib: Cali
Soo dhawoow Cali! Waxaan doortay tiro u dhaxaysa 1 iyo 50.
Waxaad haysataa 5 fursadood oo aad ku qiyaasto.

Fursadaha kuu haray: 5 | Isku-day: 0
Geli tiradaada aad qiyaastay (1-50): 25
🚫 Aad bay u WEYN TAHAY! Tiri tiro ka yar.

Fursadaha kuu haray: 4 | Isku-day: 1
Geli tiradaada aad qiyaastay (1-50): 12
📦 Aad bay u YAR TAHAY! Tiri tiro ka weyn.

Fursadaha kuu haray: 3 | Isku-day: 2
Geli tiradaada aad qiyaastay (1-50): 18

🎉 HAMBALYO Cali! WAAD GUULAYSATAY!
Tirada saxda ahayd waa: 18.
Waxaad ku heshay 3 isku-day oo keliya!
```